



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77

Somaiya Vidyavihar University



Department of Electronics Engineering

Course Name:	Object Oriented Programming	Semester:	III
Date of Performance:	___ / ___ / ____	Batch No:	Batch _A-4_
Faculty Name:		Roll No:	16015024082
Faculty Sign & Date:		Grade/Marks:	___/15

Experiment No: 5
Title: Use of Interface

Aim and Objective of the Experiment:

Learn the how to use Interfaces

COs to be achieved:

CO2: Understand concepts of Object Oriented Programming and basic characteristics of Java.

Tools used:

Theory:

Somaiya Vidyavihar University



Department of Electronics Engineering

1. (About Interfaces)

Code:

1. Write a program to design interface named *stack* with the following methods
 - a. *push* and *pop* elements from the stack
 - b. check for stack *empty* and *full* using method
2. Design class of *college* which uses two interfaces named *student* and *teacher*. Class *college* will use methods declared in the interface to *display* student and teacher personal information.

Somaiya Vidyavihar University



Department of Electronics Engineering

Output:

1



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77

Somaiya Vidyavihar University



Department of Electronics Engineering

```
import java.util.*;

interface Stack {

    void push(int x);

    int pop();

    boolean isEmpty();

    boolean isFull();

}

class ArrayStack implements Stack {

    private final int[] a;

    private int top = -1;

    ArrayStack(int capacity) {

        if (capacity <= 0) throw new IllegalArgumentException("capacity must be > 0");

        this.a = new int[capacity];

    }

    public void push(int x) {

        if (isFull()) throw new IllegalStateException("stack is full");

        a[++top] = x;

    }

    public int pop() {

        if (isEmpty()) throw new NoSuchElementException("stack is empty");

        return a[top--];

    }

    public boolean isEmpty() { return top == -1; }

    public boolean isFull() { return top == a.length - 1; }

}

public class Main {

    public static void main(String[] args) {

        Stack s = new ArrayStack(1);

        s.push(10);

        s.push(20);

        s.push(30);

        System.out.println(s.isFull());

        System.out.println(s.pop());

        System.out.println(s.pop());

        System.out.println(s.isEmpty());

        System.out.println(s.pop());

        System.out.println(s.isEmpty());

    }

}
```

Somaiya Vidyavihar University



Department of Electronics Engineering

```
/opt/homebrew/Cellar/openjdk/23.0.1/libexec/openjdk.jdk/Contents/Home/bin/ja
true
30
20
false
10
true

Process finished with exit code 0
```

2

Somaiya Vidyavihar University



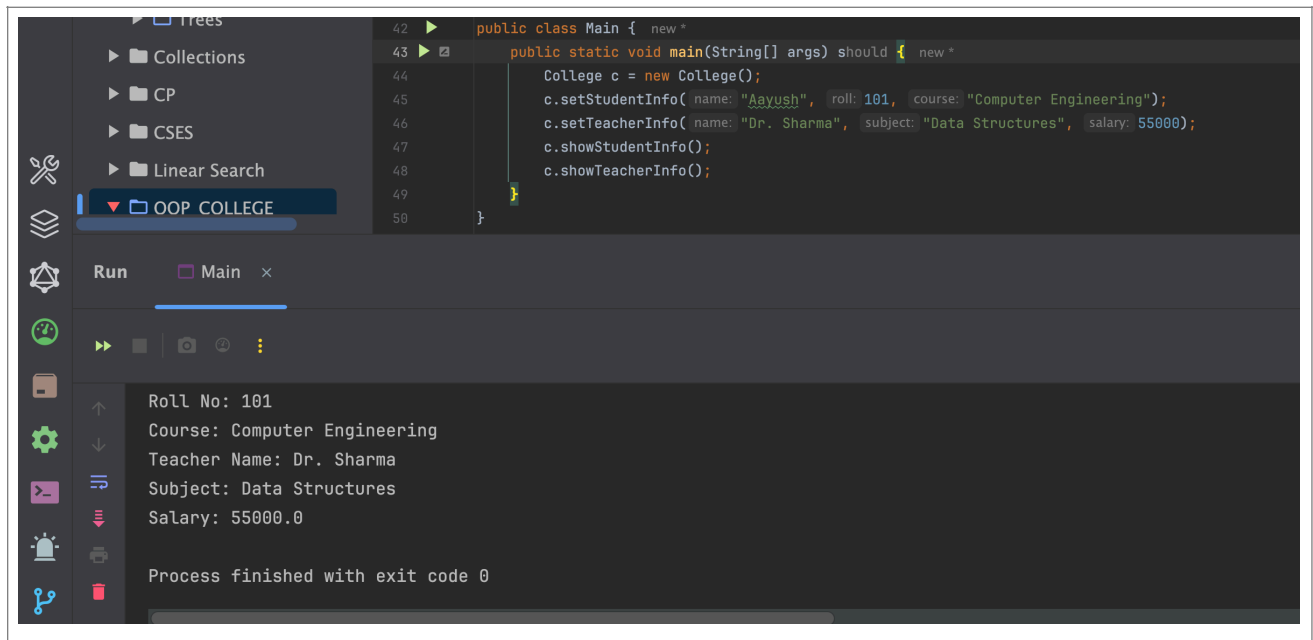
Department of Electronics Engineering

```
1 interface Student { 1 usage 1 implementation new *
2     void setStudentInfo(String name, int roll, String course); 1 usage 1 implementation new *
3     void showStudentInfo(); 1 usage 1 implementation new *
4 }
5 interface Teacher { 1 usage 1 implementation new *
6     void setTeacherInfo(String name, String subject, double salary); 1 usage 1 implementation new *
7     void showTeacherInfo(); 1 usage 1 implementation new *
8 }
9 class College implements Student, Teacher { 2 usages new *
10     private String studentName; 2 usages
11     private int rollNo; 2 usages
12     private String course; 2 usages
13     private String teacherName; 2 usages
14     private String subject; 2 usages
15     private double salary; 2 usages
16
17     public void setStudentInfo(String name, int roll, String course) { 1 usage new *
18         this.studentName = name;
19         this.rollNo = roll;
20         this.course = course;
21     }
22
23     public void showStudentInfo() { 1 usage new *
24         System.out.println("Student Name: " + studentName);
25         System.out.println("Roll No: " + rollNo);
26         System.out.println("Course: " + course);
27     }
28
29     public void setTeacherInfo(String name, String subject, double salary) { 1 usage new *
30         this.teacherName = name;
31         this.subject = subject;
32         this.salary = salary;
33     }
34
35     public void showTeacherInfo() { 1 usage new *
36         System.out.println("Teacher Name: " + teacherName);
37         System.out.println("Subject: " + subject);
38         System.out.println("Salary: " + salary);
39     }
```

Somaiya Vidyavihar University



Department of Electronics Engineering



```
42 public class Main { new *
43 public static void main(String[] args) should { new *
44     College c = new College();
45     c.setStudentInfo( name: "Aayush", roll: 101, course: "Computer Engineering");
46     c.setTeacherInfo( name: "Dr. Sharma", subject: "Data Structures", salary: 55000);
47     c.showStudentInfo();
48     c.showTeacherInfo();
49 }
50 }
```

Run Main x

Roll No: 101
Course: Computer Engineering
Teacher Name: Dr. Sharma
Subject: Data Structures
Salary: 55000.0

Process finished with exit code 0

Post Lab Subjective/Objective type Questions:



1. What is interface? How is it different from an abstract class?

-> An **interface** in Java is a special type that defines a **contract of methods** that a class must implement, but it does not contain the implementation of those methods (only their signatures).

2. Explain how multiple inheritance can be implemented using interfaces.

—>

```
interface A { void methodA(); }  
interface B { void methodB(); }  
class C implements A, B {  
    public void methodA() { System.out.println("A's method"); }  
    public void methodB() { System.out.println("B's method"); }  
}
```

3. What will be the function of the following code ? Will this code work ? If not, why ?



```
class DupValue
{
    public static void main(String[] args)
    {
        Vector v = new Vector();
        v.add("Delhi");
        v.add("Mumbai");
        v.add("Calcutta");
        v.add("Chennai");
        v.add("Delhi");

        Vector tmpVector = new Vector();
        String tmpValue;
        for (int j = 0; j <= v.size(); j++)
        {
            tmpValue = (String)v.elementAt(j);
            if(tmpValue!=null) {
                if( tmpVector.isEmpty() )
                    tmpVector.addElement(tmpValue);
                if(tmpVector.indexOf(tmpValue)==-1){
                    tmpVector.addElement(tmpValue);}}
        }
        for(int j = 0; j < tmpVector.size(); j++) {
            System.out.print(tmpVector.elementAt(j));
        }
    }
}
```

The code is meant to remove duplicate city names from the vector and print only unique ones. It won't work because the loop uses `j <= v.size()` instead of `< v.size()`, which causes an `ArrayIndexOutOfBoundsException`.

Conclusion:

In this experiment, we learned how to define and implement interfaces in Java to achieve abstraction and multiple inheritance.

By using interfaces, we were able to separate the method declarations from their implementation, making the code more modular, reusable, and easier to maintain.

Somaiya Vidyavihar University



Department of Electronics Engineering

Signature of faculty in-charge with Date: